BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. The input must be sorted.

Q2. O(n log n).

Q3. f=g+h.

Q4. Pop removes C then B; the final stack is [A,D] bottom-to-top.

You Do pages · Independent questions

Q1. Linear [2], binary [2], Dijkstra [2] with correct contexts.

Q2. Bubble $O(n^2)$, O(1) [2]; insertion $O(n^2)$,O(1) [2]; merge O(n log n),O(n) [2]; quick $O(n^2)$ worst, recursion stack depends on partitions [2].

Q3. One-off linear or indexed lookup on original unsorted input [2], sorting/indexing requires preprocessing [1], repeated sorted searches permit binary O(log n) [2], data changes may demand index maintenance [1].

Exit Ticket · Model answer

False. Preconditions and data structure matter.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 085 · Lesson 085 · Searching, Sorting and Pathfinding Comparison

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is the sorted-input rule for binary search?

Q2. State merge sort's worst-time complexity.

Q3. State f score in A*.

Q4. Push A,B,C onto a stack, Pop twice, then Push D. Give bottom-to-top order.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

suitability — How well method fits input/task.

precondition — Requirement for correct use.

complexity — Growth of time/space.

heuristic — Estimate guiding search.

New learning

Observe — Start from the problem and input: searching, ordering and weighted paths are different problem classes.

Apply — A fastest method under one set of assumptions may be invalid or less suitable on another dataset; include preprocessing and memory costs.

Evaluate — Choose using correctness, worst-case growth, memory, stability where relevant, and an independent trace on new values.

Worked example

Choose algorithm for 200 sorted pupils by ID with repeated lookups.

1. Method and result — Binary search O(log n) is valid because identifiers are sorted; if indexed dictionary is available, keyed lookup is another option.

We Do · Practise with guidance

Choose method for shortest road journey across a network with non-negative costs but no heuristic.

```
Dijkstra finds least-cost path with non-negative edge weights; do not confuse
  with array search.
```

Method and result

Dijkstra finds least-cost path with non-negative edge weights; do not confuse with array search.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Identify suitable algorithm for (a) unsorted match (b) sorted match (c) non-negative weighted shortest path. [6]

Question 2

Compare bubble, insertion, merge and quick sort worst-case time and typical auxiliary memory. [8]

Question 3

A college searches a million unsorted attendance records once and later queries sorted year-group reports repeatedly. Evaluate algorithms/data processing choices. [6]

Exit Ticket · Answer before checking

All standard algorithms can be swapped freely without checking their input assumptions.